

CSA1401 – CLASS TEST QUESTIONS

Name: Enamala Gowtham

Reg.no:192311190

Q1. Write down the translation scheme to generate code for assignment statement. List the scheme for generating three address code for the assignment statement $x = (a + b) * (c + d)$

Answer:

A translation scheme generates intermediate code by introducing temporary variables for intermediate expressions. For an assignment $E1 = E2$, the expression on the right-hand side is evaluated first and its final temporary value is assigned to the left-hand-side variable.

For $x = (a + b) * (c + d)$:

```
t1 = a + b
t2 = c + d
t3 = t1 * t2
x  = t3
```

Translation scheme:

```
S → id = E
E → E1 + E2      { E.place = newtemp(); emit(E.place = E1.place + E2.place) }
E → E1 * E2      { E.place = newtemp(); emit(E.place = E1.place * E2.place) }
E → (E1)         { E.place = E1.place }
E → id           { E.place = id.place }
S → id = E       { emit(id.place = E.place) }
```

Thus, the generated TAC contains three temporary calculations followed by the assignment to x.

Q2. Consider the following Boolean expression: $(a > b) \&\& (c < d) \parallel (e < f)$

Answer:

(i) Operator precedence and associativity: Relational operators are evaluated before logical operators. Logical AND (&&) has higher precedence than logical OR (||). Therefore the expression is interpreted as $((a > b) \&\& (c < d)) \parallel (e < f)$. AND and OR are evaluated with left associativity at the same logical level.

(ii) TAC using short-circuit evaluation:

```
L1: if a > b goto L2
    goto L5
L2: if c < d goto L4
    goto L5
L4: goto Ltrue
L5: if e < f goto Ltrue
    goto Lfalse
```

```

Ltrue:
    ...
Lfalse:
    ...

```

The first condition is evaluated first. If $a > b$ is false, the AND part is false, so evaluation can continue directly to the OR operand $e < f$. If $a > b$ is true, $c < d$ is evaluated. If both are true, the complete AND part is true and the OR expression is immediately true.

(iii) Control flow: L1 tests $a > b$; L2 tests $c < d$; L5 tests $e < f$; Ltrue and Lfalse are the final destinations. This implements short-circuit behavior without unnecessarily evaluating later conditions.

Q3. Consider the following assignment statements involving pointers: $x = *y$; $a = \&x$;

Answer:

(i) TAC:

```

t1 = *y
x  = t1
t2 = &x
a  = t2

```

(a) Pointer dereferencing ($*y$): the address stored in y is used to read the value from the referenced memory location. The value is placed in $t1$ and then assigned to x .

(b) Address-of ($\&x$): the address of x is computed and stored in $t2$, which is then assigned to pointer variable a .

(ii) Intermediate representations:

(a) Quadruples (op, arg1, arg2, result)

op	arg1	arg2	result
*	y	-	t1
=	t1	-	x
&	x	-	t2
=	t2	-	a

(b) Triples (index, op, arg1, arg2)

Index	op	arg1	arg2
0	*	y	-
1	=	(0)	x
2	&	x	-
3	=	(2)	a

(c) Indirect triples: an additional pointer table refers to the triple entries, so the order of execution can be changed by changing pointers rather than moving the triples.

```

Pointer table:
P0 → Triple 0
P1 → Triple 1
P2 → Triple 2
P3 → Triple 3

```

```

Triple table:

```

```

0: *   y   -
1: =   (0) x
2: &   x   -
3: =   (2) a

```

Q4(i). Consider: while (A < C && B > D) do if (A == 1) then C = C + 1 else while (A <= D) do A = A + B. Generate TAC.

Answer:

```

L1: if A < C goto L2
    goto L8
L2: if B > D goto L3
    goto L8

L3: if A == 1 goto L4
    goto L6

L4: t1 = C + 1
    C = t1
    goto L7

L6: if A <= D goto L9
    goto L7
L9: t2 = A + B
    A = t2
    goto L6

L7: goto L1

L8: exit

```

The outer while condition uses short-circuit evaluation: if either A < C or B > D is false, control goes to L8. The if-else statement branches through L4/L6. The inner while loop uses L6 as its condition test and L9 as its body. L7 returns to the outer-loop condition.

Q4(ii). Translate the executable statements of the C program into TAC.

Program: main(){ int i; int a[10]; i=1; while(i<=10){ a[i]=0; i=i+1; } }

Answer:

```

i = 1

L1: if i <= 10 goto L2
    goto L5

L2: t1 = i * 4
    a[t1] = 0

    t2 = i + 1
    i = t2
    goto L1

```

L5: exit

Array representation: $t1 = i * 4$ computes the byte offset when an integer occupies 4 bytes; $a[t1]$ represents the corresponding array element. If the implementation uses a different integer size, the scale factor changes accordingly. The loop is controlled by L1, and the increment operation is represented using temporary t2.

Q5. Consider: $(a < b) \parallel (c < d) \&\& (e < f)$. Using backpatching, answer the following.

Answer:

(i) Syntax-directed definition / translation scheme:

```
For  $B \rightarrow B1 \parallel B2$ :
    backpatch( $B1.falselist$ , nextquad)
     $B.truelist = merge(B1.truelist, B2.truelist)$ 
     $B.falselist = B2.falselist$ 

For  $B \rightarrow B1 \&\& B2$ :
    backpatch( $B1.truelist$ , nextquad)
     $B.truelist = B2.truelist$ 
     $B.falselist = merge(B1.falselist, B2.falselist)$ 

For  $B \rightarrow !B1$ :
     $B.truelist = B1.falselist$ 
     $B.falselist = B1.truelist$ 

For  $B \rightarrow E1 \text{ relop } E2$ :
     $B.truelist = makelist(nextquad)$ 
    emit(if  $E1 \text{ relop } E2$  goto _)
     $B.falselist = makelist(nextquad)$ 
    emit(goto _)

For  $B \rightarrow true$ :
     $B.truelist = makelist(nextquad)$ 
    emit(goto _)

For  $B \rightarrow false$ :
     $B.falselist = makelist(nextquad)$ 
    emit(goto _)
```

(ii) Because $\&\&$ has higher precedence than $\parallel$, the expression is interpreted as $(a < b) \parallel ((c < d) \&\& (e < f))$. A short-circuit TAC using backpatching is:

```
1: if  $a < b$  goto _
2: goto _

3: if  $c < d$  goto _
4: goto _

5: if  $e < f$  goto _
6: goto _

7: result = true
```

```

8: goto 10
9: result = false
10: ...

```

(iii) Backpatching lists and operations: Let $B1 = (a < b)$, $B2 = (c < d)$, and $B3 = (e < f)$.

```

For B1:
B1.truelist  = makelist(1) = {1}
B1.falselist = makelist(2) = {2}

For B2:
B2.truelist  = makelist(3) = {3}
B2.falselist = makelist(4) = {4}

For B3:
B3.truelist  = makelist(5) = {5}
B3.falselist = makelist(6) = {6}

For B2 && B3:
backpatch(B2.truelist, 5)
B23.truelist = {5}
B23.falselist = merge({4}, {6}) = {4,6}

For B1 || B23:
backpatch(B1.falselist, 3)
B.truelist  = merge({1}, {5}) = {1,5}
B.falselist = {4,6}

```

Final labeled intermediate code after backpatching:

```

1: if a < b goto TRUE
2: goto L3
3: if c < d goto L5
4: goto FALSE
5: if e < f goto TRUE
6: goto FALSE

```

```

TRUE:
    result = true
    goto END

```

```

FALSE:
    result = false

```

```

END:
    ...

```

This demonstrates makelist for creating unresolved jump lists, merge for combining lists, and backpatch for filling the correct target labels after the required destinations are known.